they also needed to be on call. Finance added policies that allowed read-only access to quarterly reports for finance analysts, but only during active reporting windows, as shown in figure 6.4. HR enforced stricter policies on access to personnel files, combining attributes like department, sensitivity level, and audit status. Each team retained autonomy over its policies while inheriting shared organizational constraints, making ABAC not just flexible but also composable and consistent across domains.

Because ABAC separates policy from identity and data systems, it scales smoothly across teams and tools. Attributes can be sourced from various systems—such as identity stores, device managers, and time services—and combined at runtime without tying policy logic to any specific application. As organizations adopt broader governance models or need to demonstrate compliance, ABAC allows them to layer and adjust rules gradually. This makes it a powerful fit for regulated industries, operational teams needing accuracy, and any enterprise aiming to make real-time, context-aware access decisions without overwhelming administrators or engineers.

6.2.5 When ABAC falls short

ABAC offers strong flexibility, but this flexibility has a cost. Since policies are written as code, they demand a level of precision and discipline that not all teams can sustain. At ACME Corp, the identity team started to face challenges as policies multiplied. Teams often reused similar logic with slight differences, leading to dozens or even hundreds of overlapping policies. Without a consistent taxonomy for attributes or clear naming conventions, it became hard to determine which policy applied to which system or why. This made understanding access decisions more complicated, especially for teams outside of engineering who lacked the tools or expertise to trace how specific attributes affected access outcomes.

ABAC also faced challenges in scenarios where access depended not just on who someone was or what device they were using, but on how they connected with others. This became clear in ACME’s Customer Collaboration platform. There, account managers needed to grant temporary access to documents, customers had to invite colleagues, and support engineers required scoped access based on account ownership. These patterns didn’t neatly fit into attribute-driven logic and were awkward to